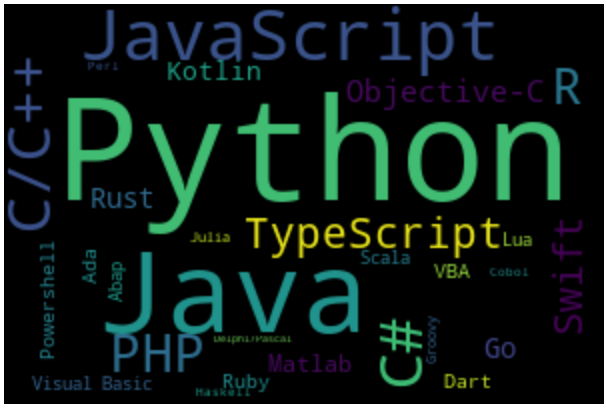
Curs 1: Introducere în Python, cu exemple

Bibliografie

1. Allen B. Downey, *Think Python, 3rd edition*, O'Reilly Media
2. Charles Severance, *Python for Everybody (PY4E)*
3. Luciano Ramalho, *Fluent Python*, ediția a doua
4. Al Sweigart, *Automate the Boring Stuff with Python*
5. Hans Petter Langtangen, *A Primer on Scientific Programming with Python*, Springer-Verlag, 2016, accesibilă prin e-nformation
6. Google's Python Class
7. Tutoriale online: Python Programmer

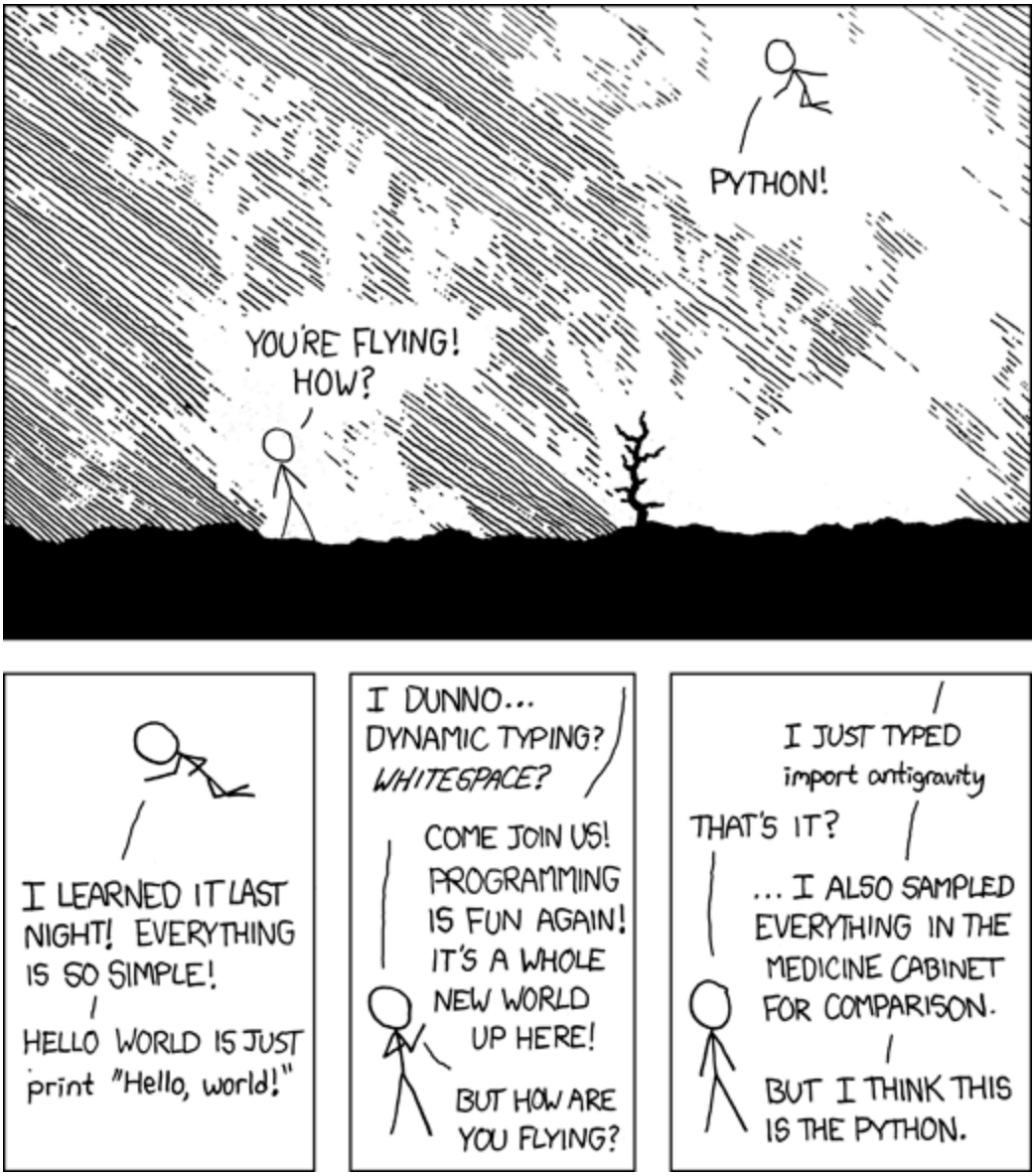
Limbajul Python

- Limbaj open source, gratuit
- În octombrie 2026: primul limbaj ca popularitate în TIOBE Index, devansând C, C++, Java
- În aceeași lună, indexul PYPL raportează Python pe locul întâi



(Sursa: http://pypl.github.io/PYPL.html, octombrie 2026)

- Versiunea stabilă actuală de Python este 3.x; ultima variantă se găsește pe site-ul oficial Python; Python 2.x e considerat mort din 2020
- Se poate instala manual sau folosind UV/Poetry pentru management de medii de lucru
- Python este limbaj interpretat, dinamic și puternic tipizat
- Permite programare imperativă, orientată pe obiect, funcțională, metaprogramare
- Există o paletă largă de biblioteci deja implementate pentru Python, ușurând considerabil dezvoltarea de prototipuri sau chiar de aplicații comerciale larg răspândite.



(Sursa: https://xkcd.com/353/)

- Peste 900000 de pachete dezvoltate
- Python este în adoptare crescândă pentru proiecte comerciale:
  - Full Stack Python
  - How to Use Python for Enterprise Application Development in 2025
  - Is Python Right for Your Business in 2026? Strategic Pros and Cons
  - 10 Companies Using Python in 2026 & 5 Real Use Cases Driving Career Opportunities
  - 25 Leading Companies Using Python in 2025

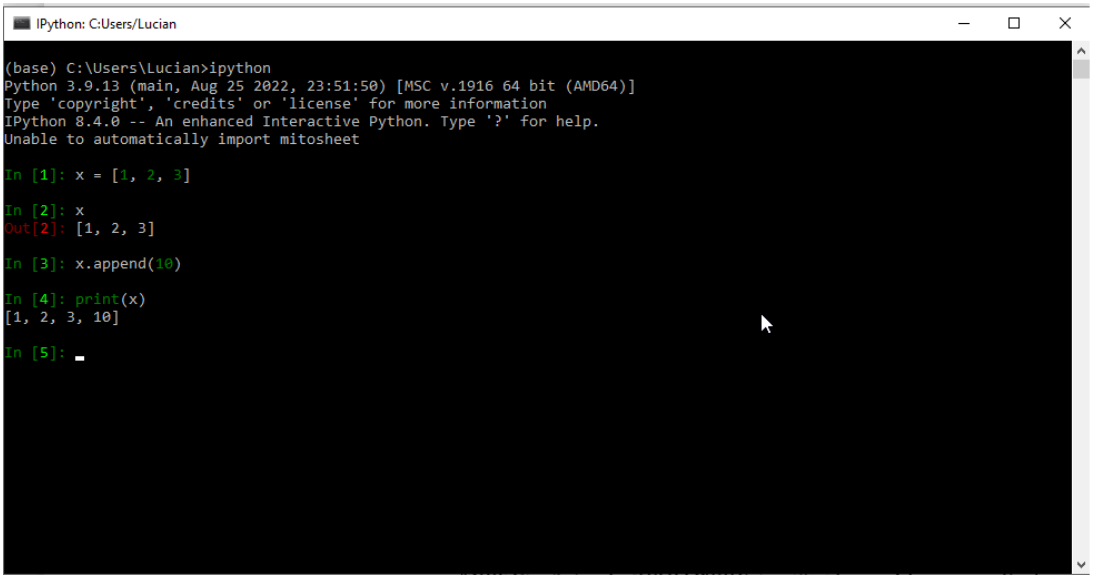
De ce Python?

Iată câteva motive solide pentru a utiliza Python, conform The unexpected effectiveness of Python in science:

1. Limbajul vine "cu toate bateriile incluse": multitudinea de biblioteci îl face perfect pentru o gamă largă de proiecte. Cercetătorii care îl folosesc activează în astronomie, fizică, bioinformatică, chimie, științe cognitive etc. - și pentru toate aceste domenii există biblioteci care simplifică mult dezvoltarea de prototipuri. Prin comparație, limbaje tradiționale precum C/C++/Java vin cu mai puțin suport disponibil, iar uneori includerea unui pachet/biblioteci poate fi o experiență consumatoare de timp.
2. Capacitatea de interoperare cu alte limbaje este iarăși un plus. Există punți de comunicare (bridges) care permit apelul de cod Python din alte limbaje sau invers. Desigur, această facilitate se regăsește și în alte limbaje.
3. Natura dinamică a limbajului poate fi inteligent speculată: o funcție poate să preia o colecție de valori organizată în diverse moduri și câteva linii de cod funcționează la fel de bine peste toate acestea.
4. Python încurajează un stil de lucru experimental, via Read, Evaluate, Print, and Loop (REPL): poate e nevoie de câteva încercări pentru a ajunge la instrucțiunile potrivite pentru o anumită funcționalitate. Încercarea unei alte instrucțiuni nu necesită recompilarea sau rerularea codului anterior, ci vine în completare - până când ajungi la ce dorești să obții.

Moduri de utilizare

1. Codul poate fi scris într-un fișier text cu extensia py. Lansarea codului se face cu comanda `python nume_script.py`
2. Se poate folosi interpretorul python sau ipython, în care modul de lucru este REPL. REPL permite o prototipizare rapidă și evoluția codului în mod incremental. Interpretorul ipython este o variantă îmbunătățită a interpretorului python.



   Mai multe detalii pentru lucrul cu interpretorul ipython se găsesc în bibliografie: debug, magic commands, lucru cu interpretorul de comenzi etc.
3. Jupyter notebook - codul este scris în browser, cu suport pentru completare automată de cod. Fișierele rezultate sunt cu extensia ipynb. O listă de notebooks se găsește aici.

Resurse de lucru cu Jupyter notebooks aici

4. Folosind diverse framework-uri (*i.e.* Flask), codul Python poate fi folosit pentru crearea de pagini Web, REST endpoints - sau mai simplu, funcțiile definite în Jupyter notebook pot fi expuse ca servicii web REST etc.

Variabile, tipuri de date

Clasica afișare de mesaj "Hello world" se obține cu *funcția* `print()`:

In [1]:
```
print("Hello world!")
# sau: print('Hello world')
```

```
Hello world!
```

Delimitarea șirurilor de caractere se poate face cu apostroafe sau ghilimele.

Variabilele nu se declară în prealabil împreună cu tipul lor; tipul unei variabile este determinat în momentul execuției instrucțiunii care o referă. Natura unei variabile se deduce din valoarea care îi este asociată:

In [2]:
```
x = 3  # x e variabila de tip intreg
y = "abcd"  # y este variabila de tip sir de caractere
# se obisnuieste ca numele compuse ale variabilelor sa fie despartite prin _:
nume_complet = "Popescu Ion"  # string
x = [1, 2, 3]  # acum x este o lista
```

Python este case sensitive: Nume_complet și nume_complet referă variabile diferite. Atribuirea se face folosind semnul egal, așa cum s-a văzut mai sus.

Tipurile concrete ale variabilelor pot fi aflate la rulare:

In [3]:
```
# functia type
print(type(x))
print(type(y))
```

```
<class 'list'>
<class 'str'>
```

Variabilelor li se pot da nume începând cu caracter Unicode sau _, continuând cu caractere, _ sau cifre. Următoarele cuvinte rezervate nu pot fi folosite ca nume de variabile:

In [4]:
```
import keyword

print(f"Cuvinte cheie: {keyword.kwlist}")
```

```
Cuvinte cheie: ['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await', 'b
reak', 'class', 'continue', 'def', 'del', 'elif', 'else', 'except', 'finally', 'fo
r', 'from', 'global', 'if', 'import', 'in', 'is', 'lambda', 'nonlocal', 'not', 'or',
'pass', 'raise', 'return', 'try', 'while', 'with', 'yield']
```

Mai multe despre cuvinte cheie: Python Keywords.

Tipuri numerice

In [5]:
```
height = 1.79
weight = 78
body_mass_index = weight / height**2  # la numitor este inaltimea la patrat
print("Indicele de masa corporala este: ", body_mass_index)
```

```
Indicele de masa corporala este:  24.343809494085704
```

Pentru tipurile întregi, operatorii utilizabili sunt:

- +
- -
- *
- / (împărțire cu rezultat în virgulă mobilă)
- // (împărțire întreagă)
- % (modulo)
- ** (ridicare la putere)

In [6]:
```
print(7 / 3)
print(7 // 3)
print(7 % 3)
print(7**3)
```

```
2.3333333333333335
2
1
343
```

În Python 3 există un singur tip `int`, cu precizia de reprezentare limitată de memoria alocată procesului:

In [7]:
```
import math  # despre module, in partea a doua a cursului

big_number = math.factorial(100)
print("100!=", big_number)
```

```
100!= 93326215443944152681699238856266700490715968264381621468592963895217599993229915608941463976156518286253697920827223758251185210916864000000000000000000000000
```

Pentru tipurile în virgulă mobilă, operatorii sunt aceiași, dar cu rezultat de tip număr în virgulă mobilă. Dacă într-o expresie un operand este în virgulă mobilă, operația returnează valoare în virgulă mobilă. Exemple:

In [8]:
```
print(5.0 - 5)
print(7.5 // 2)
print(7.5 % 2)
print(
    1.12 % 1
```

```
)  # partea fractionara; atentie la erori de reprezentare a numerelor "reale" in bi
# impartirea intreaga pentru numere negative:
print(-7 // 2)  # -4
```

```
0.0
3.0
1.5
0.1200000000000001
-4
```

Numerele se pot compara folosind operatorii:

- <, >, <=, >=
- == (egalitate), != (diferit)

Există suport nativ pentru numere complexe:

In [9]:
```
z = 1 + 4j
print(z)
print(type(z))
print(z.conjugate())
z2 = z * z.conjugate()
print(z2)
```

```
(1+4j)
<class 'complex'>
(1-4j)
(17+0j)
```

Sunt admise conversii între tipuri numerice:

In [10]:
```
a = 3.997
b = int(a)
print(b)
print(float(b))
```

```
3
3.0
```

Operarea cu numere în virgulă mobilă este tributară unor erori de reprezentare:

In [11]:
```
print(f"0.1 + 0.2 = {0.1 + 0.2}")
print(f"0.1 + 0.2 == 0.3: {0.1 + 0.2 == 0.3}")
print(f"1.0-0.9 = {1.0 - 0.9}")
print(f"1.0 - 0.9 == 0.1: {1.0 - 0.9 == 0.1}")
print(f"1.0-0.9 - 0.1= {1.0 - 0.9 - 0.1}")
```

```
0.1 + 0.2 = 0.30000000000000004
0.1 + 0.2 == 0.3: False
1.0-0.9 = 0.09999999999999998
1.0 - 0.9 == 0.1: False
1.0-0.9 - 0.1= -2.7755575615628914e-17
```

În limbajul Python, pentru a compara două numere reale, se poate folosi funcția math.isclose, care acceptă două argumente (numerele reale de comparat) și un argument

opțional abs_tol, care reprezintă toleranța absolută la eroare. Dacă nu se specifică abs_tol, se folosește o valoare implicită de 1e-09.

In [12]:
```
# Documentatie math.isclose: https://docs.python.org/3/library/math.html#math.isclo
print(f"0.1 + 0.2 ~ 0.3: {math.isclose(0.1 + 0.2, 0.3)}")
print(f"1.0 - 0.9 ~ 0.1: {math.isclose(1.0 - 0.9, 0.1)}")
print(
    math.isclose(1.0 - 0.9 - 0.1, 0.0, abs_tol=1e-12)
)  # abs_tol aici e necesar, altfel se returneaza False, deoarece eroarea de reprez
```

```
0.1 + 0.2 ~ 0.3: True
1.0 - 0.9 ~ 0.1: True
True
```

In [13]:
```
# Alte exemple: https://www.tutorialspoint.com/python/python_numbers.htm
```

Șiruri de caractere

Șirurile de caractere se delimitează cu apostroafe sau cu ghilimele. Pentru literalii de tip șir de caractere, există posibilitatea de a scrie valori care nu necesită secvențe escape:

In [14]:
```
a = r"adresa \\server\share\...."  # remarcam prefixul r
print(a)

# the hard way...
a = "adresa \\\\server\\share\\...."
print(a)

# caractere Unicode: implicit, in Python 3 toate stringurile sunt Unicode
a = "ĂÎÂȘȚ aăîâșț"
print(a)
```

```
adresa \\server\share\....
adresa \\server\share\....
ĂÎÂȘȚ aăîâșț
```

..sau stringuri multilinie, folosind delimitare cu trei ghilimele sau trei apostroafe:

In [15]:
```
versuri = """If you can keep your head when all about you
    Are losing theirs and blaming it on you,
If you can trust yourself when all men doubt you,
    But make allowance for their doubting too;
If you can wait and not be tired by waiting,
    Or being lied about, don’t deal in lies,
Or being hated, don’t give way to hating,
    And yet don’t look too good, nor talk too wise:

If you can dream—and not make dreams your master;
    If you can think—and not make thoughts your aim;
If you can meet with Triumph and Disaster
    And treat those two impostors just the same;
If you can bear to hear the truth you’ve spoken
    Twisted by knaves to make a trap for fools,
Or watch the things you gave your life to, broken,
```

```
    And stoop and build ’em up with worn-out tools:"""
print(versuri)
```

```
If you can keep your head when all about you
    Are losing theirs and blaming it on you,
If you can trust yourself when all men doubt you,
    But make allowance for their doubting too;
If you can wait and not be tired by waiting,
    Or being lied about, don’t deal in lies,
Or being hated, don’t give way to hating,
    And yet don’t look too good, nor talk too wise:

If you can dream—and not make dreams your master;
    If you can think—and not make thoughts your aim;
If you can meet with Triumph and Disaster
    And treat those two impostors just the same;
If you can bear to hear the truth you’ve spoken
    Twisted by knaves to make a trap for fools,
Or watch the things you gave your life to, broken,
    And stoop and build ’em up with worn-out tools:
```

Șirurile de caractere în Python sunt imuabile:

In [16]:
```
sir = "abc"
# eroare
# sir[0] = "A"  # nu se poate modifica un string, deoarece este imutabil
```

Șirurile de caractere se pot concatena cu + (și generează alte șiruri) și suportă următoarele operații și funcții:

In [17]:
```
sir = "Ana " + "are " + "mere"
print(sir)
```

```
Ana are mere
```

In [18]:
```
print("Lungimea sirului: ", len(sir))
print("Primul element din sir: ", sir[0])
print("Ultimul element din sir: ", sir[len(sir) - 1])
print("Ultimul element, in stil Python: ", sir[-1])
print("Penultimul element din sir, in stil Python: ", sir[-2])  # !!!
```

```
Lungimea sirului:  12
Primul element din sir:  A
Ultimul element din sir:  e
Ultimul element, in stil Python:  e
Penultimul element din sir, in stil Python:  r
```

In [19]:
```
# Conversia unei variabile de tip non-string in string:
x = 3
mesaj = "numarul este " + str(x)
print(mesaj)
```

```
numarul este 3
```

In [20]:
```
# metode si operatii posibile pe obiect string:
print(sir.lower())
```

```
print("slicing:", sir[4:7])
print(sir.find("nu are"))
print(sir.replace("Ana", "Ion"))
```

```
ana are mere
slicing: are
-1
Ion are mere
```

In [21]:
```
multi_ana = "Ana" * 10
print(multi_ana)
```

```
AnaAnaAnaAnaAnaAnaAnaAnaAnaAna
```

In [22]:
```
cuvantul_pere_este_in_sir = "pere" in sir
print(cuvantul_pere_este_in_sir)
cuvantul_pere_nu_este_in_sir = "pere" not in sir
print(cuvantul_pere_nu_este_in_sir)
```

```
False
True
```

In [23]:
```
tokens = sir.split(" ")
print(type(tokens))
print(tokens)
print(",".join(["Ana", "Vasile", "Dana", "Ion"]))
```

```
<class 'list'>
['Ana', 'are', 'mere']
Ana,Vasile,Dana,Ion
```

Demnă de menționat este funcția `eval`, care preia un string de caractere și returnează un obiect rezultat prin evaluare:

In [24]:
```
a = 3
b = 4
expresie = "(a+b)/(a**2 + b**2 + 1)"
print(eval(expresie))
```

```
0.2692307692307692
```

Atenție, însă: `eval` execută orice expresie Python validă, nu doar calcule aritmetice. Aceasta este o problemă de securitate. De exemplu, codul de mai jos duce la ștergerea de fișiere, rulând cu drepturile procesului:

In [25]:
```
expresie = input("Introduceti expresia: ")
# scriem la consola:  __import__('os').system('rm -rf ~')
eval(expresie)  # rulează comanda în sistemul de operare
```

Out[25]: 4

Funcția `eval` nu se folosește niciodată pe șiruri care vin de la utilizator, din fișiere sau din rețea. Dacă șirul conține doar un literal (număr, listă, dicționar, tuplu), se folosește `ast.literal_eval`, care acceptă exclusiv literali și aruncă o excepție `ValueError` pentru orice altceva:

In [26]:
```
import ast  # se importa un modul; detalii in partea a doua a cursului

my_list = ast.literal_eval("[1, 2, {'a': 3}]")  # [1, 2, {'a': 3}]
print(f"my_list: {my_list}")
# decomentati pentru a vedea eroarea ce apare atunci cand se incearca evaluarea unu
# ast.literal_eval("__import__('os')")  # ValueError: malformed node
```

```
my_list: [1, 2, {'a': 3}]
```

Recomandăm citirea documentației oficiale pentru tipul de date string.

Python conține tipul `bool`, util pentru reprezentarea valorilor de adevăr `True` și `False`:

In [27]:
```
x = True
print(type(x))
```

```
<class 'bool'>
```

Se pot face conversii de la tipuri numerice la bool și viceversa: valoarea 0 este asociată lui `False`, orice non-zero lui `True`. Invers, `True` se traduce în 1 și `False` în 0:

In [28]:
```
x = bool(-1)
print("-1 ca bool: ", x)
x = bool(0.0)
print("0.0 ca bool: ", x)
b = True
print("True ca int:", int(b))
b = False
print("False ca int:", int(b))
```

```
-1 ca bool:  True
0.0 ca bool:  False
True ca int: 1
False ca int: 0
```

Operatorii aplicabili pe valori bool sunt:

In [29]:
```
print(True and False)
print(True or False)
print(not False)
# xor
a, b = True, False
a_xor_b = a ^ b
print(a_xor_b)
```

```
False
True
True
True
```

Liste

În interiorul unei liste se pot pune oricâte elemente, nu neapărat de același tip. Elementele se despart prin virgulă; capetele listei sunt marcate prin paranteze drepte:

In [30]:
```
lista = [10, 20, 30, 40]
print("Lungimea listei este", len(lista))
print("Tipul listei este", type(lista))
# Lista eterogena: string si int
lista2 = ["Ana", 234]
```

```
Lungimea listei este 4
Tipul listei este <class 'list'>
```

Elementele unei liste se accesează pe bază de indici, începând de la 0 și terminând cu `len(lista)-1`. Ultimul element se poate referi cu indicele '-1':

In [31]:
```
lista = [10, 20, 30]
print("Ultimul element este:", lista[-1])
print("Penultimul element este:", lista[-2])
```

```
Ultimul element este: 30
Penultimul element este: 20
```

O listă se poate obține prin repetarea unei construcții de un număr dorit de ori:

In [32]:
```
lista = [1, 2, 3] * 5  # efect similar demonstrat anterior pe string 'Ana'
print(lista)
```

```
[1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3]
```

Putem prelua porțiuni întregi din listă, nu doar de pe poziții individuale, folosind 'decuparea' (slicing), sub forma: lista[k:l]. Va rezulta o listă formată din elementele lista[k], lista[k+1], ..., lista[l-1] (se remarcă marginea din dreapta a șirului: nu e l, ci l-1). Dacă se dorește ca ordinea elementelor să fie inversată, se folosește lista[l:k:-1]; elementul de indice l va fi inclus în rezultat, elementul de indice k se va omite:

In [33]:
```
lista = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
print(lista)
sliced = lista[3:8]
print(sliced)
sliced_reversed = lista[8:3:-1]
print(sliced_reversed)
```

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
[4, 5, 6, 7, 8]
[9, 8, 7, 6, 5]
```

In [34]:
```
lista = [10, 20, 30, 40]
```

Formele acceptate pentru slicing sunt:

- a[start:end] # elementele de la indicele `start` la `end-1` (inclusiv)
- a[start:] # elementele de la indicele `start` (inclusiv) până la final

- a[:end] # elementele de la început până la `end-1` (inclusiv)
- a[:] # o copie a întregii liste

În special ultima formă este interesantă: se obține o clonă a listei a. În lipsa acestui mecanism, două variabile care indică spre aceeași listă pot duce la modificări mutual vizibile:

In [35]:
```
# aliasing
a = [1, 2, 3]
b = a
print("a=", a)
print("b=", b)
a[0] *= -1
print("Dupa modificare via a: b=", b)
# a si b refera aceeasi zona de memorie
```

```
a= [1, 2, 3]
b= [1, 2, 3]
Dupa modificare via a: b= [-1, 2, 3]
```

In [36]:
```
# copiere de lista
a = [1, 2, 3]
b = a[:]
print("a=", a)
print("b=", b)
a[0] *= -1
print("Dupa modificare: a=", a)
print("Dupa modificare: b=", b)
# Concluzia: a si b refera zone de memorie diferite
```

```
a= [1, 2, 3]
b= [1, 2, 3]
Dupa modificare: a= [-1, 2, 3]
Dupa modificare: b= [1, 2, 3]
```

Copierea unei liste se poate face și cu metoda `copy`:

In [37]:
```
# copiere de lista cu metoda copy
a = [1, 2, 3]
b = a.copy()
print("a=", a)
print("b=", b)
a[0] *= -1
print("Dupa modificare: a=", a)
print("Dupa modificare: b=", b)
# Concluzia: a si b refera zone de memorie diferite
```

```
a= [1, 2, 3]
b= [1, 2, 3]
Dupa modificare: a= [-1, 2, 3]
Dupa modificare: b= [1, 2, 3]
```

Totuși, ambele copieri fac copiere superficială: listele imbricate rămân partajate.

In [38]:
```
# demonstratie shallow copy
```

```
lista_sursa = [1, 2, [3, 4], [5, 6, 7]]
lista_copie = lista_sursa.copy()
print("lista_sursa=", lista_sursa)
lista_copie[2][0] *= -1
print("Dupa modificare: lista_sursa=", lista_sursa)
```

```
lista_sursa= [1, 2, [3, 4], [5, 6, 7]]
Dupa modificare: lista_sursa= [1, 2, [-3, 4], [5, 6, 7]]
```

Dacă se dorește copierea "în profunzime" (deep copy), se folosește funcția `deepcopy` din modulul `copy`:

In [39]:
```
import copy

lista_sursa = [1, 2, [3, 4], [5, 6, 7]]
lista_copie = copy.deepcopy(lista_sursa)
print("lista_sursa=", lista_sursa)
lista_copie[2][0] *= -1
print("Dupa modificare: lista_sursa=", lista_sursa)
# observam ca lista sursa nu e modificata, deoarece s-a folosit deepcopy
```

```
lista_sursa= [1, 2, [3, 4], [5, 6, 7]]
Dupa modificare: lista_sursa= [1, 2, [3, 4], [5, 6, 7]]
```

Listele pot fi concatenate:

In [40]:
```
a = [1, 2, 3]
b = [10, 20, 30]
c = a + b
print(c)
```

```
[1, 2, 3, 10, 20, 30]
```

La o listă se pot adăuga elemente:

In [41]:
```
a = [1, 2, 3]
a.append(100)
print(a)
```

```
[1, 2, 3, 100]
```

Adăugările repetate via append au cost amortizat $O(1)$ (pentru cost amortizat, a se vedea capitolul de `Amortized Analysis` din `Introduction to Algorithms`, Cormen et al.). Totuși, unii programatori preferă prealocarea listei, dacă se știe de la început numărul exact (sau maxim) de elemente:

In [42]:
```
lista_prealocata = [None] * 10
# ... cod...
lista_prealocata[3] = -100
print(lista_prealocata)
```

```
[None, None, None, -100, None, None, None, None, None, None]
```

Sortarea unei liste se face cu funcția `sorted`, care opțional poate să preia:

- un parametru numit `reverse` de tip boolean, care precizează dacă se face sortare descrescătoare
- un parametru numit `key`, care face referință la o funcție ce determină politica de sortare

In [43]:
```
# Sortare crescatoare
a = [5, 1, 4, 3]
print(sorted(a))
```

```
[1, 3, 4, 5]
```

In [44]:
```
# Sortare descrescatoare
print(sorted(a, reverse=True))
```

```
[5, 4, 3, 1]
```

In [45]:
```
# Sortare cu criteriu specificat
siruri = ["ccc", "aaaa", "d", "bb"]
print(sorted(siruri, key=len))
```

```
['d', 'bb', 'ccc', 'aaaa']
```

Funcția `sorted` returnează o nouă listă, lăsând pe cea originară nemodificată. Dacă se dorește ca sortarea să se facă în cadrul listei originare, se va folosi metoda `sort` a tipului listă. Metoda `sort` returnează `None`.

In [46]:
```
siruri = ["ccc", "aaaa", "d", "bb"]
siruri.sort()
print(siruri)
```

```
['aaaa', 'bb', 'ccc', 'd']
```

Ștergerea unui element dintr-o listă, de pe o poziție (indice) specificat se face cu `del`:

In [47]:
```
lista = [10, 20, 30]
del lista[1]
print(lista)
```

```
[10, 30]
```

Testarea existenței unui element într-o listă se face cu operatorul `in`:

In [48]:
```
lista = ["Ana", "are", "mere"]
print("portocale" in lista)
```

```
False
```

Minimul și maximul unei liste se obțin cu funcțiile `min` respectiv `max`.

Alte metode aparținând tipului list sunt exemplificate mai jos:

In [49]:
```
lista = [10, 20, 30, 10]
# De cate ori apare elementul 10 in lista?
print("De cate ori apare elementul 10 in lista", lista.count(10))
# Care e primul index in care un anumit element apare in lista?
```

```
print("Care e primul index in care un anumit element apare in lista?", lista.index(
# daca pentru metoda index se specifica un element care nu exista, se arunca except
# print(lista.index(100000))
# ValueError: 100000 is not in list
# Inserare elementului 100 pe indexul 2 in lista:
lista.insert(2, 100)
print("Inserare elementului 100 pe indexul 2 in lista", lista)
# Inversarea ordinii elementelor dintr-o lista, in-place
lista.reverse()
print("Inversarea ordinii elementelor dintr-o lista, in-place", lista)
# Stergerea unui element din lista, prima aparitie
lista.remove(10)
print("Stergerea unui element din lista, prima aparitie", lista)
lista.clear()
print("Lista a fost golita:", lista)
```

```
De cate ori apare elementul 10 in lista 2
Care e primul index in care un anumit element apare in lista? 1
Inserare elementului 100 pe indexul 2 in lista [10, 20, 100, 30, 10]
Inversarea ordinii elementelor dintr-o lista, in-place [10, 30, 100, 20, 10]
Stergerea unui element din lista, prima aparitie [30, 100, 20, 10]
Lista a fost golita: []
```

Pentru testarea faptului că toate elementele (respectiv: măcar un element al) unei liste de valori boolene sunt cu valoarea `True`, se va folosi funcția `all` (respectiv: `any`).

In [50]:
```
lista = [True, False, True]
print("Macar unul e True:", any(lista))
print("Toate sunt True:", all(lista))
```

```
Macar unul e True: True
Toate sunt True: False
```

Dacă lista este goală, funcția `all` returnează `True`, iar `any` - `False`.

In [51]:
```
print("Toate sunt True:", all([]))
print("Macar unul e True:", any([]))
```

```
Toate sunt True: True
Macar unul e True: False
```

O metodă deosebit de elegantă de procesare a elementelor unei liste este prin list comprehension, ce se va prezenta în partea a doua a introducerii în Python.

Operatorii `==` și `is`: exemplificare pentru liste

Python are doi operatori similari, dar verifică lucruri diferite:

- `a == b` verifică **egalitatea valorilor**: au `a` și `b` același conținut?
- `a is b` verifică **identitatea**: sunt `a` și `b` unul și același obiect în memorie?

În Python variabilele sunt nume care referă obiecte. Două nume pot referi două obiecte diferite cu același conținut sau pot referi același obiect. Identitatea unui obiect se poate afla

cu funcția `id()`, iar `a is b` este echivalent cu `id(a) == id(b)`. În implementarea de Python curentă, funcția `id` returnează adresa de memorie a argumentului.

In [52]:
```
a = [1, 2, 3]
b = [1, 2, 3]  # o lista noua, cu acelasi continut
c = a  # inca un nume pentru lista referita de a

print("a == b:", a == b)
print("a is b:", a is b)
print("a == c:", a == c)
print("a is c:", a is c)
print("id(a), id(b), id(c):", id(a), id(b), id(c))
```

```
a == b: True
a is b: False
a == c: True
a is c: True
id(a), id(b), id(c): 1995734302976 1995734259904 1995734302976
```

Operatorii negați sunt `!=` (valori diferite) și `is not` (obiecte diferite). Se folosește `x is not None`, nu `not x is None`.

`is` se folosește și pentru comparație cu `None`:

In [53]:
```
rezultat = None
# ... instructiuni...
if rezultat is None:
    print("Nu exista inca un rezultat")

# se prefera scrierea urmatoare:
if not rezultat:
    print("Nu exista inca un rezultat")
```

```
Nu exista inca un rezultat
Nu exista inca un rezultat
```

Tupluri

În timp ce o listă permite modificarea conținutului său, un tuplu reprezintă o colecție ordonată imuabilă. Elementele se separă cu virgulă, capetele tuplului se marchează de regulă cu paranteze rotunde, sau pot lipsi.

In [54]:
```
tuplu1 = ("informatica", 3, "dimineata")
tuplu2 = "chimie", 2, "seara"
print(tuplu1)
```

```
('informatica', 3, 'dimineata')
```

In [55]:
```
print(tuplu1[0])
```

```
informatica
```

In [56]:
```
print(tuplu1[-1])
```

```
dimineata
```

In [57]:
```
print(len(tuplu1))
```

```
3
```

In [58]:
```
tuplu_gol = ()
print(len(tuplu_gol))
```

```
0
```

In [59]:
```
la_tuplul_cu_doar_o_valoare_trebuie_adaugata_virgula_la_sfarsit = (42,)
print(len(la_tuplul_cu_doar_o_valoare_trebuie_adaugata_virgula_la_sfarsit))
```

```
1
```

Elementele unui tuplu pot fi preluate individual în variabile:

In [60]:
```
print(tuplu1)
a, b, c = tuplu1
print(f"{a=}")
print(f"{b=}")
print(f"{c=}")
```

```
('informatica', 3, 'dimineata')
a='informatica'
b=3
c='dimineata'
```

In [61]:
```
# Tuplurile se pot concatena
tuplu1 + tuplu2
```

Out[61]:
```
('informatica', 3, 'dimineata', 'chimie', 2, 'seara')
```

In [62]:
```
# Convertirea unei liste in tuplu:
lista = [1, 2, 3, 4, 5]
tuplu = tuple(lista)
print(type(tuplu))
print(tuplu)
```

```
<class 'tuple'>
(1, 2, 3, 4, 5)
```

Dicționare

Dicționarele sunt colecții de asocieri între chei și valori. Colecția de tip dicționar se demarchează cu acolade. Elementele se accesează specificând între paranteze drepte valoarea cheii.

In [63]:
```
geografie = {}  # dictionar gol
geografie = {"Romania": "Bucuresti", "Serbia": "Belgrad"}
print("Lungimea este:", len(geografie))
key = "Romania"
print("Valoarea pentru cheia", key, "este", geografie[key])
geografie["Grecia"] = "Atena"  # adaugare de pereche cheie-valoare in dictionar
# daca se foloseste o cheie care exista, valoarea asociata va fi suprascrisa cu cea
```

```
geografie["Grecia"] = "Athens"
# accesarea folosind o cheie invalida duce la eroare (KeyError)
# geografie['Franta']
# pentru a evita eroare la rulare, se poate folosi metoda get; daca cheia nu e gasi
print(geografie.get("Franta"))
# daca se doreste returnarea unei valori prestabilite in cazul lipsei cheii, metoda
print(geografie.get("Franta", "<Nu exista in dictionar>"))
```

```
Lungimea este: 2
Valoarea pentru cheia Romania este Bucuresti
None
<Nu exista in dictionar>
```

Colecțiile cheilor și a valorilor unui dicționar se determină cu metoda `keys`, respectiv `values`:

In [64]:
```
print("Chei:", geografie.keys())
print("Valori:", geografie.values())
```

```
Chei: dict_keys(['Romania', 'Serbia', 'Grecia'])
Valori: dict_values(['Bucuresti', 'Belgrad', 'Athens'])
```

Oricare din colecții poate fi transformată într-o listă prin apelul constructorului `list()` care poate prelua o colecție oarecare:

In [65]:
```
print(list(geografie.keys()))
print(list(geografie.values()))
```

```
['Romania', 'Serbia', 'Grecia']
['Bucuresti', 'Belgrad', 'Athens']
```

Ștergerea unei chei din dicționar, împreună cu valoarea asociată, se face cu instrucțiunea `del`:

In [66]:
```
del geografie["Grecia"]
print(geografie)
```

```
{'Romania': 'Bucuresti', 'Serbia': 'Belgrad'}
```

Testarea faptului că o anumită cheie se află într-un dicționar se face cu operatorul `in`:

In [67]:
```
print("Grecia" in geografie)
```

```
False
```

Golirea unui dicționar se face, precum la alte tipuri colecție cu metoda `clear`:

In [68]:
```
# geografie.clear()
```

Colecția perechilor (cheie, valoare) a unui dicționar se obține cu metoda `items()`:

In [69]:
```
print(geografie.items())
print(list(geografie.items()))
```

```
dict_items([('Romania', 'Bucuresti'), ('Serbia', 'Belgrad')])
[('Romania', 'Bucuresti'), ('Serbia', 'Belgrad')]
```

Dacă la un dicționar se dorește adăugarea perechilor (cheie-valoare) ale altui dicționar se folosește metoda `update`:

In [70]:
```
geografie_asia = {"China": "Beijing", "India": "New Delhi"}
geografie.update(geografie_asia)
print(geografie)
```

```
{'Romania': 'Bucuresti', 'Serbia': 'Belgrad', 'China': 'Beijing', 'India': 'New Delh
i'}
```

Clonarea unui dicționar se face cu metoda `copy`:

In [71]:
```
geografie_copie = geografie.copy()
geografie_copie["India"] = "New----Delhi"
print(geografie["India"], ",", geografie_copie["India"])
```

```
New Delhi , New----Delhi
```

O prezentare detaliată și bine structurată pentru dicționare se găsește aici.

Alte colecții

Se folosește foarte frecvent funcția `range` care produce o secvență de numere. Formele de utilizare sunt:

- `range(n)` produce colecția 0, 1, ..., n-1
- `range(start, stop)` produce colecția start, start+1, ..., stop-1
- `range(start, stop, step)` produce secvența `start, start+step, start+2*step, ... k`, cu ultimul termen `k` determinat automat în funcție de semnul lui `step`:
  - pentru `step` pozitiv, `k` este cel mai mare întreg mai mic decât `stop` care se obține prin adăugarea unui multiplu întreg al pasului `step` la start
  - pentru `step` negativ, `k` e cel mai mic număr strict mai mare decât `stop`

In [72]:
```
for i in range(2, 10):
    print(i, end=" ")
```

```
2 3 4 5 6 7 8 9
```

In [73]:
```
for i in range(2, 10, 3):
    print(i, end=" ")
```

```
2 5 8
```

In [74]:
```
for i in range(10, 2, -2):
    print(i, end=" ")
```

```
10 8 6 4
```

Funcția `enumerate` pornește de la o colecție și dă acces simultan atât la indicele elementului din colecție, cât și la elementul curent:

In [75]:
```
lista1 = ["a", "b", "c"]
for i, item in enumerate(lista1):
    print(i, item)
```

```
0 a
1 b
2 c
```

Funcția `zip` permite 'împerecherea' a două liste

In [76]:
```
lista2 = ["m", "n", "p"]

for pereche in zip(lista1, lista2):
    print(pereche)
```

```
('a', 'm')
('b', 'n')
('c', 'p')
```

Un alt tip de date frecvent utilizat este tipul mulțime (set), cu prezentare in extenso aici.

Instrucțiuni, comentarii

Blocuri de instrucțiuni

Instrucțiunile se scriu de regulă câte una pe linie. Dacă pentru o instrucțiune prea lungă se dorește continuarea ei pe linia următoare, se pune la finalul liniei caracterul \ și se continuă pe rând nou. PEP 8 recomandă scrierea între paranteze:

In [77]:
```
# a = 1 + 2 + \
# 3 + 4
# preferabil, in Python, este sa se foloseasca paranteze pentru a grupa expresia:
a = (1 + 2 + 3
     + 4)
print(a)
```

```
10
```

Pentru colecții, continuarea pe rând nou nu necesită caracter backslash:

In [78]:
```
lista_lunga = [1, 2, 3, 4, 5, 6]
```

Se pot scrie mai multe instrucțiuni pe o linie, despărțindu-se cu caracterul ';'. Acest stil însă e nerecomandat

In [79]:
```
print("1")
print("2")
```

```
1
2
```

Un bloc de instrucțiuni va folosi aceeași indentare. Convenția PEP 8 e de 4 spații pentru indentare. Un astfel de bloc de instrucțiuni se termină cu prima linie care nu e indentată în același stil ca și blocul.

In [80]:
```
if 1 + 1 == 3:
    print("Nu se afiseaza")
print("Linia aceasta nu face parte din blocul corespunzator instructiunii if")
```

```
Linia aceasta nu face parte din blocul corespunzator instructiunii if
```

Comentarii

Comentariile sunt pe o singură linie, începând de la caracterul # până la finalul liniei.

In [81]:
```
a, b, c = 1, 2, 3  # coeficientii ecuatiei de gradul 2: ax^2 + bx + c = 0
delta = b**2 - 4 * a * c  # se calculeaza discriminantul ecuatiei de gradul 2
```

Atribuirea

Atribuirea a fost exemplificată mai sus. Mai avem variantele:

In [82]:
```
# Atribuire multipla cu o valoare
a = b = c = 3
```

In [83]:
```
# Atribuire multipla cu mai multe valori simultan
a, b = 2, 3
print(a, b)
```

```
2 3
```

Ultima formă poate fi speculată astfel: dacă `a` și `b` sunt două variabile și se dorește interschimbarea valorilor lor, atunci putem scrie:

In [84]:
```
print("Inainte de interschimbare: ", a, b)
a, b = b, a
print("Dupa interschimbare: ", a, b)
```

```
Inainte de interschimbare:  2 3
Dupa interschimbare:  3 2
```

Instrucțiunea `if`

Formele uzuale sunt:

```
if expresie:
    bloc 1
else:
    bloc 2
```

Partea `else` poate să lipsească. Blocurile de instrucțiuni ce urmează după `if` și `else` sunt indentate. După expresia lui `if` și după `else` se pun două puncte.

In [85]:
```
var1 = 100
var2 = 200
if var1 > var2:
    print("In if - Got a true expression value")
    print(var1)
else:
    print("In else - Got a false expression value")
    print(var2)
```

```
In else - Got a false expression value
200
```

O instrucțiune `if` poate conține multiple teste, folosind `elif`:

```
if expression1:
   statement(s)
elif expression2:
   statement(s)
elif expression3:
   statement(s)
else:
   statement(s)
```

Se evaluează în ordine condițiile `expression1`, `expression2` etc. până la prima care e găsită ca fiind adevărată; blocul ei este executat și restul se ignoră

In [86]:
```
var = 100
if var == 200:
    print("1 - Got a true expression value")
    print(var)
elif var == 150:
    print("2 - Got a true expression value")
    print(var)
elif var == 100:
    print("3 - Got a true expression value")
    print(var)
else:
    print("4 - Got a false expression value")
    print(var)
```

```
3 - Got a true expression value
100
```

Instrucțiunea `if` se poate folosi și inline, în forma:

```
expression_if_true if condition else expression_if_false
```

Exemplu:

In [87]:
```
a, b = 3, 4
max_a_b = a if a > b else b
```

Pentru calcul unei valori de tip boolean, deseori expresia inline `if` e redundantă:

In [88]:
```
printCNP = "5678901234567"
# antipattern
# gen_masculin = True if int(CNP[0]) % 2 == 1 else False
gen_masculin = int(printCNP[0]) % 2 == 1
print("gen masculin:", gen_masculin)
```

```
gen masculin: True
```

Instrucțiunea `match`

Începând cu Python 3.10 există instrucțiunea `match`, care compară o valoare cu o serie de tipare (*patterns*), în ordinea în care sunt scrise. Se execută blocul primului tipar care se potrivește, iar restul sunt ignorate. Forma generală este:

```
match expresie:
    case tipar_1:
        bloc 1
    case tipar_2:
        bloc 2
    case _:
        bloc implicit
```

Tiparul `_` se potrivește cu orice valoare; poate să lipsească. Spre deosebire de instrucțiunea `switch` din C/C++/Java, nu există "cădere" de la un `case` la următorul și nu este nevoie de `break`.

In [89]:
```
cod_raspuns = 404
match cod_raspuns:
    case 200:
        print("OK")
    case 404:
        print("Resursa nu a fost gasita")
    case 500:
        print("Eroare pe server")
    case _:
        print("Cod necunoscut:", cod_raspuns)
```

```
Resursa nu a fost gasita
```

Mai multe valori pot fi grupate în același `case` folosind `|`, pe post de "sau":

In [90]:
```
zi = "sambata"
match zi:
    case "sambata" | "duminica":
        print("Weekend")
    case "luni" | "marti" | "miercuri" | "joi" | "vineri":
        print("Zi lucratoare")
    case _:
        print("Zi invalida:", zi)
```

```
Weekend
```

Se poate folosi și fenomenul de capturare a porțiunii unei secvențe:

In [91]:
```
punct = (0, 5)
match punct:
    case (0, 0):
        print("Originea")
    case (0, y):
        print("Pe axa Oy, la y =", y)
    case (x, 0):
        print("Pe axa Ox, la x =", x)
    case (x, y):
        print("Punct oarecare:", x, y)
```

```
Pe axa Oy, la y = 5
```

În `case (0, y)`, valoarea `0` trebuie să coincidă, iar `y` este o variabilă care primește ce se află pe poziția respectivă = *capturare*. Un tipar de secvență se potrivește doar dacă numărul de elemente coincide. Tiparul `(x, y)` acceptă atât tuplul `(3, 4)`, cât și lista `[3, 4]` - importantă e structura și nu tipul colecției. Șirurile de caractere nu sunt tratate ca secvențe în `match`.

Cu `*nume` se captează toate elementele rămase, într-o listă (posibil goală). Exemplul de mai jos interpretează comenzi text, împărțite în cuvinte cu `split`:

In [92]:
```
comenzi = [
    "ajutor",
    "incarca date.csv",
    "antreneaza 10 0.01",
    "sterge a.txt b.txt c.txt",
    "iesire acum",
]

for linie in comenzi:
    match linie.split():
        case ["ajutor"]:
            print("Comenzi disponibile: incarca, antreneaza, sterge")
        case ["incarca", fisier]:
            print("Se incarca fisierul", fisier)
        case ["antreneaza", epoci, rata]:
            print("Antrenare:", int(epoci), "epoci, rata de invatare", float(rata))
        case ["sterge", *fisiere]:
            print("Se sterg", len(fisiere), "fisiere:", fisiere)
        case _:
            print("Comanda necunoscuta:", linie)
```

```
Comenzi disponibile: incarca, antreneaza, sterge
Se incarca fisierul date.csv
Antrenare: 10 epoci, rata de invatare 0.01
Se sterg 3 fisiere: ['a.txt', 'b.txt', 'c.txt']
Comanda necunoscuta: iesire acum
```

Tiparele se pot aplica și pentru dicționare:

In [93]:
```
configuratie = {"optimizator": "adam", "rata_invatare": 0.001, "beta1": 0.9}

match configuratie:
    case {"optimizator": "sgd", "rata_invatare": rata}:
        print("SGD cu rata de invatare", rata)
    case {"optimizator": "adam", "rata_invatare": rata, **rest}:
        print("Adam cu rata de invatare", rata, "si parametri suplimentari", rest)
    case {"optimizator": nume}:
        print("Optimizator necunoscut sau configuratie incompleta:", nume)
    case _:
        print('Configuratie invalida: lipseste cheia "optimizator"')
```

```
Adam cu rata de invatare 0.001 si parametri suplimentari {'beta1': 0.9}
```

Verificarea tipului pentru tipare

Un tipar de forma `int()`, `float()`, `str()`, `list()` se potrivește dacă valoarea are tipul respectiv. Cu `as` se poate da un nume valorii potrivite:

In [94]:
```
valori = [42, 3.14, True, "text", [1, 2], None]

for v in valori:
    match v:
        case bool():
            print(repr(v), "-> bool")
        case int() | float() as numar:
            print(repr(numar), "-> numar")
        case str():
            print(repr(v), "-> sir de caractere")
        case list():
            print(repr(v), "-> lista de lungime", len(v))
        case None:
            print(repr(v), "-> lipsa valorii")
```

```
42 -> numar
3.14 -> numar
True -> bool
'text' -> sir de caractere
[1, 2] -> lista de lungime 2
None -> lipsa valorii
```

Ciclare: for

Ciclarea cu `for` este folosită pentru a itera o instrucțiune sau un bloc de instrucțiuni peste o colecție cunoscută.

In [95]:
```
for i in range(0, 3):
    print("i=", i)
```

```
i= 0
i= 1
i= 2
```

In [96]:
```
# Exemplu: calculul factorialului unui numar
n = 100
p = 1
for i in range(1, n + 1):
    p *= i
print(n, "!=", p)
```

```
100 != 933262154439441526816992388562667004907159682643816214685929638952175999932299156089414639761565182862536979208272237582511852109168640000000000000000000000000
```

In [97]:
```
# %%timeit
# n = 100
# list_numbers = [str(i) for i in range(1, n+1)]
# expression = '*'.join(list_numbers)
# print(eval(expression))
```

In [98]:
```
lista_nume = ["Ana", "Dan", "Rares", "Dana"]
for nume in lista_nume:
    print(nume)
```

```
Ana
Dan
Rares
Dana
```

In [99]:
```
geografie = {"Romania": "Bucuresti", "Serbia": "Belgrad", "Grecia": "Atena"}
for key in geografie:
    print(geografie[key])
```

```
Bucuresti
Belgrad
Atena
```

In [100…
```
# iterare cu doua elemente, peste colectie de perechi
for key, value in geografie.items():
    print("Capitala pentru", key, "este", value)
```

```
Capitala pentru Romania este Bucuresti
Capitala pentru Serbia este Belgrad
Capitala pentru Grecia este Atena
```

In [101…
```
for index, element in enumerate(lista_nume):
    print(index, element)
```

```
0 Ana
1 Dan
2 Rares
3 Dana
```

In [102…
```
# enumerare peste doua colectii 'impreunate'
lista_ani = [20, 21, 22, 23]
for nume, ani in zip(lista_nume, lista_ani):
    print(nume, "are", ani, "ani")
```

```
Ana are 20 ani
Dan are 21 ani
Rares are 22 ani
Dana are 23 ani
```

Se poate forța ieșirea dintr-un ciclu `for` cu instrucțiunea `break`. Se poate sări peste o parte din blocul unui ciclu `for` și trece la ciclarea următoare folosind `continue`:

In [103…
```python
for i in range(100):
    if i > 5:
        break
    print(i)
```

```
0
1
2
3
4
5
```

In [104…
```python
for i in range(10):
    if i % 2 == 0:
        continue
    print(i)
```

```
1
3
5
7
9
```

O formă aparte a instrucțiunii `for` în Python este aceea în care la final se pune `else`: dacă nu s-a executat `break` în corpul ciclului, atunci se execută blocul de după `else`:

In [105…
```python
for n in range(2, 10):
    for x in range(2, n):
        if n % x == 0:
            print(n, "equals", x, "*", n // x)
            break
    else:
        # loop fell through without finding a factor
        print(n, "is a prime number")
```

```
2 is a prime number
3 is a prime number
4 equals 2 * 2
5 is a prime number
6 equals 2 * 3
7 is a prime number
8 equals 2 * 4
9 equals 3 * 3
```

Ciclare: while

Folosind `while` se ciclează peste un bloc de instrucțiuni atâta timp cât o anumită condiție este adevărată:

```
while test_expression:
    body of while
```

In [106…

```
n = 10

# initialize sum and counter
my_sum = 0
i = 1

while i <= n:
    my_sum = my_sum + i
    i = i + 1  # update counter

# print the sum
print("The sum is", my_sum)
```

```
The sum is 55
```

Se poate ca o instrucțiune `while` să se termine cu `else`, al cărei bloc se execută când expresia de test devine falsă, dar numai dacă nu s-a executat un `break`.